SC24: International Conference for High Performance Computing, Networking, Storage and Analysis | 979-8-3503-5291-7/24/$31.00 ©2024 IEEE | DOI: 10.1109/SC41406.2024.00018

ECOLIFE: Carbon-Aware Serverless Function Scheduling for Sustainable Computing

Yankai Jiang
Northeastern University
jiang.yank@notheastern.edu

Rohan Basu Roy
Northeastern University
basuroy.r@northeastern.edu

Baolin Li
Northeastern University
li.baol@northeastern.edu

Devesh Tiwari
Northeastern University
d.tiwari@northeastern.edu

***Abstract*—This work introduces ECOLIFE, the first carbon-aware serverless function scheduler to co-optimize carbon footprint and performance. ECOLIFE builds on the key insight of intelligently exploiting multi-generation hardware to achieve high performance and lower carbon footprint. ECOLIFE designs multiple novel extensions to Particle Swarm Optimization (PSO) in the context of serverless execution environment to achieve high performance while effectively reducing the carbon footprint.**

***Keywords*—Serverless Computing, Sustainable Computing, Cloud Computing**

I. INTRODUCTION

Motivation and Goals of ECOLIFE: Carbon footprint is increasingly becoming one of the most important measures of sustainability of large-scale computing systems. Due to the growing demand for computing in datacenters, the carbon footprint of these large-scale systems is rising [1]–[11]. Carbon dioxide (CO_2) and other greenhouse gases are emitted during manufacturing datacenter hardware (termed as *embodied* carbon footprint), and also during execution of applications on the hardware (termed as *operational* carbon footprint). The embodied carbon footprint is amortized over the lifetime of the hardware, and the operational carbon footprint depends on the energy consumption of the hardware and the *carbon intensity* of the power grid that provides energy to the datacenter.

As detailed in Sec. II, we observe that datacenter hardware from different generations (old and new hardware) has different proportions of embodied and operational carbon footprints, and combining hardware from different generations has the potential of jointly minimizing both application runtime and carbon footprint. In fact, this indirectly opens up the opportunity of potentially extending the lifetime of older hardware for higher environmental sustainability of large-scale computing systems. ECOLIFE leverages this observation and opportunity in designing a scheduling solution for serverless computing. ECOLIFE *aims to make serverless computing sustainable and high-performant by performing carbon footprint-aware scheduling of serverless functions on hardware from different generations.*

Serverless computing and challenges in carbon-aware serverless scheduling: Serverless computing is gaining wider adoption as a paradigm of cloud computing due to several attractive features like a higher level of resource abstraction from end-users, auto-scaling of resources, and a pay-as-you-go billing model [12], [13]. Due to these advantages, there is increasing interest in introducing the serverless computing model to the HPC community and workflows [14]–[18], along with several related efforts in the parallel system's community [19]–[24].

To make serverless computing high-performant, service providers keep functions alive in the memory of servers so that they are not affected by a start-up overhead, also referred to as the *cold start* of functions. Keeping functions alive consumes resources and energy, which translates to a keep-alive carbon footprint. The summation of the keep-alive carbon footprint and the carbon footprint during execution constitutes the total carbon footprint of a function. Since older hardware often has lower embodied carbon [25], it can be beneficial to keep functions alive in older hardware but suffer from performance degradation during execution (Sec. II). Newer hardware is usually more energy efficient, and hence, results in lower operational carbon — representing a trade-off between different types of carbon footprint and performance (Sec. II).

Furthermore, different serverless functions need to be *kept alive* for different amounts of time depending on a function's arrival probability. Moreover, the carbon intensity of a power grid varies with time, which has an impact on the operational carbon footprint. *Since the characteristics and invocation patterns of production serverless functions and the carbon intensity vary with time, this makes carbon-aware function scheduling challenging.*

ECOLIFE's Key Contributions: ECOLIFE makes the following key contributions.

I. ECOLIFE is *a novel high-performance and carbon-aware serverless scheduler* that exploits hardware of different generations to improve the sustainability of computing systems (*exploiting the lifetime extension of older-generation hardware*) while achieving high performance. To the best of our knowledge, this is the first work that focuses on reducing the carbon footprint of serverless computing.

SC24, November 17-22, 2024, Atlanta, Georgia, USA
979-8-3503-5291-7/24/$31.00 ©2024 IEEE
Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:50:19 UTC from IEEE Xplore. Restrictions apply.

II. ECOLIFE introduces novel extensions to the Particle Swarm Optimization (PSO) technique in the context of serverless scheduling. The novel design and implementation of PSO extensions include (a) *perception-response* mechanism to adapt in the dynamic serverless environment, and (b) *function warm pool adjustment* mechanism to intelligently prioritize function keep-alive time and location among multi-generation hardware, in response to varying memory requirements.

III. ECOLIFE is evaluated using widely-used serverless function invocation trace from Microsoft Azure cloud [26], and is shown to consistently perform close to the theoretically-optimal (ORACLE) solution using multiple different generations of hardware and is robust under different scenarios. Our evaluation indicates that ECOLIFE is consistently within 7.7% and 5.5% points from ORACLE in terms of service time and carbon footprint, respectively. ECOLIFE is available at https://zenodo.org/records/10976139.

II. BACKGROUND

Serverless function keep-alive in cloud computing. The serverless computing model abstracts the cloud computing infrastructure for users, allowing them to upload their code to be executed as stateless functions. In the serverless computing model, cloud providers manage user functions as container images and orchestrate the underlying hardware resources without a need for user intervention. Upon invocation, a function's image is loaded into the server for execution. To enhance efficiency, after execution, the function remains in the server memory for a certain period. The duration during which the function is kept alive in memory is termed the *keep-alive time*, and is controlled by the cloud provider. Keeping functions alive in the memory decreases the chances of a *cold start* of a function, potentially eliminating the need to reload the function into memory. If a function is re-invoked post the keep-alive period, it incurs a *cold start overhead* that requires loading the function in the memory. If a function is re-invoked before the keep-alive period, it receives a *warm start*. The *service time* of a function is comprised of its cold start overhead (zero in the case of a warm start) plus the execution time of the function. Given that the execution times for typical production serverless functions can be comparable to the cold start overhead [27], [28], optimizing keep-alive time for serverless functions is an important design consideration.

Carbon footprint of computing systems. The carbon footprint encompasses both embodied carbon and operational carbon. Embodied carbon refers to the emissions associated with manufacturing and packaging computer hardware [29], such as that from foundries like TSMC. Since this occurs only once, the share of embodied carbon for a traditional (non-serverless) application is proportional to the execution time of the application relative to the lifespan of the device [1], [2].

Indeed, as the rapid development in the lithography process continues in hardware manufacturing, advanced hardware has improved capabilities. However, it often comes with larger die sizes, increased core counts, and expanded memory sizes [30], [31]. Hence, the manufacturing process for such newer-generation hardware often has a higher embodied carbon footprint compared to the older-generation hardware. These carbon emissions generated during manufacturing contribute to the embodied carbon footprint of the hardware and are taken into account throughout its lifespan.

Unlike embodied carbon footprint, operational carbon footprint refers to the emissions originating from the electricity supplied by grid operators to power the computing infrastructure. It is quantified as the product of the grid's carbon intensity (gCO_2/kWh) and energy usage (kWh). Here, carbon intensity denotes the amount of carbon dioxide emitted per unit of generated energy, and it varies over time. For a traditional (non-serverless) application, the operational carbon footprint includes energy consumed during the execution.

Carbon footprint estimation for a serverless function. In contrast to traditional non-serverless functions, the overall carbon footprint of a serverless function is calculated for all three periods: keep-alive period, duration of potential cold-start, and execution time (the first two periods are serverless-specific). The carbon footprint estimation in serverless is composed of *embodied carbon footprint estimation* and *operational carbon footprint estimation*. The embodied and operational carbon footprint of a serverless function accounts for the carbon footprint generated by both the CPUs and DRAMs [1], [2], [9]. Below, we briefly describe how carbon footprint is estimated – with the acknowledgment that the carbon footprint estimation of serverless functions is non-trivial and has many complex interactions, but the model described below captures the first-order principles and effects.

First, for the embodied carbon footprint, the attribution of embodied carbon is different during different phases (e.g., keep-alive period and execution time) due to differences in the amount of resources being used. The embodied carbon footprint estimation of DRAM and CPU is accounted for the usage proportion attributed to function f. The embodied carbon footprint per unit of time of a DRAM is calculated by dividing the total embodied carbon of the DRAM (EC_{DRAM}) by its lifetime (LT_{DRAM}), and multiplied with the memory usage ratio – $\frac{M_f}{M_{\text{DRAM}}}$. here, M_f is the memory size of the function f and M_{DRAM} is the size of DRAM. Then, the embodied carbon of DRAM with keep-alive time k and service time S_f can be modeled as:

$$\text{DRAM Embodied CO}_2 = \frac{S_f + k}{LT_{\text{DRAM}}} \cdot \frac{M_f}{M_{\text{DRAM}}} \cdot EC_{\text{DRAM}}$$

During the service period, the entire CPU is assigned to serverless execution. However, during the keep-alive period, one CPU core is preserved to keep the serverless function alive. The embodied carbon per core of a CPU is determined by dividing the total embodied carbon footprint of the CPU (EC_{CPU}) by the number of cores (Core_{num}). Therefore, the formal expression for the embodied carbon of the CPU can be written as:

$$\text{CPU Embodied CO}_2 = \frac{S_f}{LT_{\text{CPU}}} \cdot EC_{\text{CPU}} + \frac{k}{LT_{\text{CPU}}} \cdot \frac{EC_{\text{CPU}}}{\text{Core}_{\text{num}}}$$

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:50:19 UTC from IEEE Xplore. Restrictions apply.

TABLE I. *Multi-generation Hardware Pairs Examples.*

Pair	Old/New	CPU Model (Year)	DRAM Model (Year)
Pair_A	A_{OLD}	Intel Xeon E5-2686 (2016)	Micron-512 (2018)
	A_{NEW}	Intel Xeon Platinum 8252C (2020)	Samsung-192 (2019)
Pair_B	B_{OLD}	Intel Xeon Platinum 8124M (2017)	Micron-192 (2018)
	B_{NEW}	Intel Xeon Platinum 8252C (2020)	Samsung-192 (2019)
Pair_C	C_{OLD}	Intel Xeon Platinum 8275L (2019)	Samsung-192 (2019)
	C_{NEW}	Intel Xeon Platinum 8252C (2020)	Samsung-192 (2019)

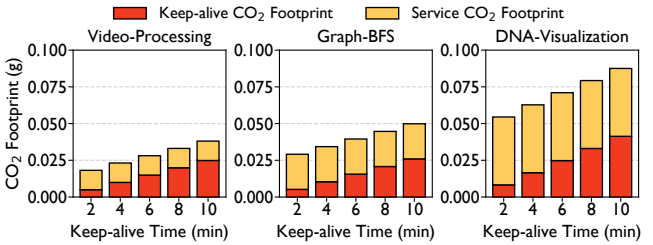

Fig. 1. *The carbon footprint (carbon footprint during keeping-alive and service) for three serverless functions for different keep-alive periods. The contribution of carbon footprint during the keep-alive period toward the overall carbon footprint is significant, esp. as the keep-alive period increases.*

The embodied carbon footprint of hardware is already incurred during manufacturing, but it must be considered and accounted for during the operational period too. Similar to how energy consumption is tracked, the distribution and attribution of the embodied carbon footprint among different applications must be carefully accounted for to inform future planning and potential resource usage.

Second, the operational carbon footprint includes executing serverless functions during invocation, in addition to the energy required to maintain it in memory during keep-alive. The operational carbon footprint of DRAM can be estimated by multiplying the real-time carbon intensity (CI) with the energy consumption of DRAM ($E_{\text{DRAM}}^{\text{Service}} + E_{\text{DRAM}}^{\text{Keep-alive}}$) during both the service period and keep-alive period. Note that the operational carbon footprint of DRAM incurs a carbon footprint based on the function's share of the overall operational carbon footprint of DRAM. Therefore, the memory usage ratio – $\frac{M_f}{M_{\text{DRAM}}}$ is multiplied to estimate the operational carbon footprint generated by DRAM of a function. The estimation of the CPU's operational carbon footprint is similar to the estimation of embodied carbon. The entire CPU is assigned to serverless function execution, but during the keep-alive period, only one CPU core is used to keep the function alive. These estimations can be formally expressed as:

$$\text{DRAM Operational CO}_2 = \frac{M_f}{M_{\text{DRAM}}} \cdot (E_{\text{DRAM}}^{\text{Service}} + E_{\text{DRAM}}^{\text{Keep-alive}}) \cdot \text{CI}$$

$$\text{CPU Operational CO}_2 = (E_{\text{CPU}}^{\text{Service}} + \frac{E_{\text{CPU}}^{\text{Keep-alive}}}{\text{Core}_{\text{num}}}) \cdot \text{CI}$$

We acknowledge that a universally accepted methodology is not widely established for the carbon footprint estimation of serverless functions. The approach outlined here is one intuitive method for modeling carbon footprints. Because each period of serverless computing is unique, separately considering each period provides an easy interpretation of the carbon footprint for serverless functions. Incorporating second-level effects (such as storage) can be modeled as an extension by adding the proportional carbon footprint of storage. Our described model does not necessarily favor ECOLIFE and is only used to demonstrate that carbon savings are possible while achieving high performance. ECOLIFE primarily focuses on CPU-based systems, which are commonly used in serverless environments [26]. While trends in our motivational observations in Sec. III apply to GPUs, too, because of multi-generational trade-offs, we do not directly focus on GPUs. ECOLIFE can be adapted for multi-generation GPUs using the GPU-specific carbon footprint model and measurement.

Table I shows three old-generation / new-generation pairs to demonstrate that the motivation and key ideas behind ECOLIFE are not restricted to a single pair, and benefits can be observed over different pairs (Sec. VI confirms this quantitatively). While one cannot practically evaluate all possible multiple generation pairs, entries in Table I were selected to capture three different types of generations (with one, two, and four years of gap representing different lifetimes of the hardware) and where accurate embodied carbon footprint data is available. We anticipate that hardware upgrades can happen in a one to five year timeline, and ECOLIFE demonstrates how it can be used to leverage the prior generation of hardware to achieve both high performance and a low carbon footprint.

III. MOTIVATION

Observation. *Serverless functions generate a significant carbon footprint during their keep-alive period — which is unique compared to the traditional non-serverless computing model, where functions are not kept alive in memory in anticipation of an actual invocation.*

First, we measure the carbon footprint generated during the keep-alive period and the service time of different serverless functions from SeBS benchmark [28] on A_{NEW} (Table I). Fig. 1 shows the trends for three representative functions: video processing, graph search, and DNA visualization. Other functions demonstrate a similar trend, but these functions were selected for motivation as they represent diverse characteristics in terms of computational and memory requirements, and also represent the core of many algorithms.

From Fig. 1, we observe that as the keep-alive period increases, the keep-alive carbon footprint also rises due to the increased embodied and operational carbon emissions. Consequently, its proportion in the total carbon footprint becomes higher. For example, when the keep-alive time is increased from 2 minutes to 10 minutes, the keep-alive carbon footprint of function Graph-BFS has increased from previously constituting 18% of the total carbon footprint to now 52% of the total carbon footprint. Fig. 1 also shows that *the carbon footprint during the keep-alive period can often be higher than the carbon footprint during the actual execution* – this is because the keep-alive period (typically multiple minutes) is often orders of magnitude longer than the execution time (often millisecond to a few seconds).

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:50:19 UTC from IEEE Xplore. Restrictions apply.

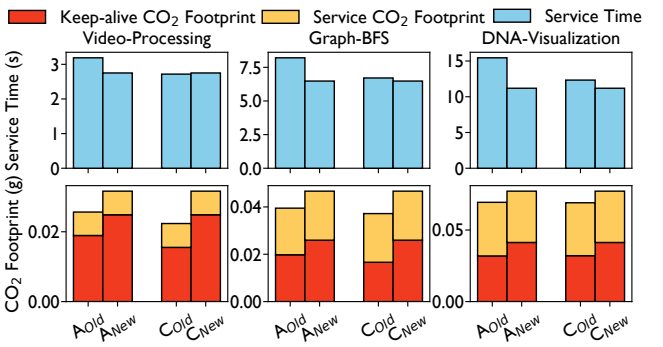

Fig. 2. *The serverless functions can incur a lower overall carbon footprint if kept alive and executed on older-generation hardware due to lower keep-alive carbon footprint (e.g., A_{OLD} vs. A_{NEW}), but they can suffer from performance degradation. However, the impact on performance can be relatively small for some functions with significant savings in carbon footprint (e.g., Graph-BFS on C_{OLD} vs. C_{NEW}). The keep-alive period is the same and constant (10 minutes) for all cases.*

Opportunity. *The use of relatively older-generation hardware, which inherently has a lower embodied carbon footprint, opens the opportunity to lower the carbon footprint during the keep-alive period.*

We observe this opportunity via comparing the overall service time and carbon footprint across various serverless workloads, using two pairs of older-newer hardware pair as illustrated in Fig. 2 (Both pair A and C are selected for demonstration). We found that while leveraging relatively older-generation hardware lowers the carbon footprint during the keep-alive period, unfortunately, it results in significant performance (execution time) degradation. For example, as shown in Fig. 2, considering executing video processing on A_{OLD} and A_{NEW}, respectively, and keeping the function alive for 10 minutes, using A_{OLD} to keep the function alive compared to using A_{NEW} can save 23.8% of carbon footprint. However, the execution time of the function increases by 15.9%. This is because, as expected, older hardware often yields slower performance for many workloads. However, this leads to interesting trade-offs where leveraging older hardware's extended lifetime for carbon footprint benefits competes with the execution time metric.

However, recall that for serverless functions, the metric for performance is service time (not execution time alone). Service time is the sum of the execution time and cold-start overhead (if the function was not warm or kept alive in memory at the time of invocation). *Interestingly, a lower carbon footprint on older hardware during the keep-alive period enables us to afford a longer keep-alive period on older hardware compared to newer hardware under the same or lower carbon footprint budget. This indirectly results in higher chances of warm starts and hence, potentially lower service time even when using older hardware.* However, navigating this trade-off is challenging because the magnitude of the trade-off varies across different functions and hardware generations.

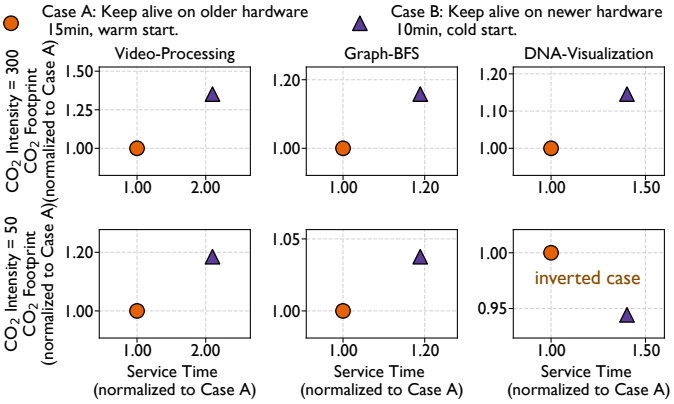

Fig. 3. *Trade-off between carbon footprint and service time: A longer keep-alive period on older-generation hardware can potentially reduce both service time and carbon footprint, but the magnitude and feasibility depend on function characteristics and carbon intensity.* ***Case A****: keep alive for 15 mins on C_{OLD}, receive a warm start (no cold start overhead) but slower execution.* ***Case B****: keep alive for 10 mins on C_{NEW}, and suffer from cold start but faster execution time.*

Challenge. *To effectively exploit the opportunity identified earlier (trade-off between carbon footprint and service time), one needs to intelligently determine the keep-alive period for different functions on different generations of hardware – the optimal periods can be different for different functions and vary over time.*

In Fig. 3, we perform a comparative experiment to measure the corresponding service time and overall carbon footprint with two testing scenarios (Case A and Case B, described in Fig. 3) on two generations of hardware (C_{OLD} vs C_{NEW}) under two carbon intensity (Carbon Intensity = 50, Carbon Intensity = 300). The results provide strong experimental evidence for the previously discussed opportunity. For example, in Fig. 3 (top row with Carbon Intensity = 300), when the video-processing function is kept alive in memory for a longer time (15 mins) and executed on older hardware (C_{OLD}) with warm start, it leads to a 52.3% saving in service time and a 14.9% saving in carbon footprint compared to utilizing shorter keep-alive periods (10 mins) on newer hardware (C_{NEW}) with cold start. This is true for Graph-BFS and DNA visualization functions, too.

We show that utilizing older hardware for extended keep-alive time could potentially reduce carbon emissions while maintaining high performance, because of the increasing possibility of warm starts (hence, eliminating the cold start overhead). Intuitively, when carbon intensity is high, it is worth keeping the function alive on the old-generation hardware (by incurring relatively lower embodied carbon) rather than experiencing the cold start on the new-generation hardware and incurring a high operational carbon footprint during the cold-start period. Essentially, we attempt to eliminate high operational carbon footprint during the cold start on newer hardware by being able to keep the function alive on older

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:50:19 UTC from IEEE Xplore. Restrictions apply.

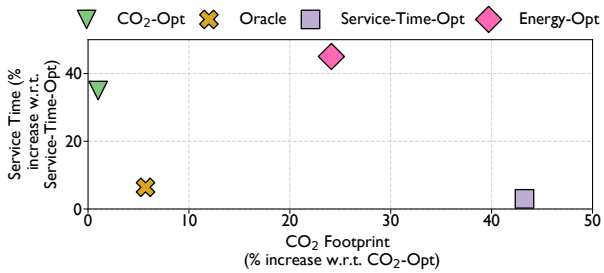

Fig. 4. *CO_2-OPT, SERVICE-TIME-OPT and ENERGY-OPT are far from the ORACLE. There is an opportunity to jointly optimize the overall carbon footprint and service time, but it is challenging to exploit. (A_{OLD} vs A_{NEW})*

hardware – which incurs lesser embodied carbon and reduces chances of cold start (and hence, better service too).

However, the magnitude of this benefit can be reduced or absent in some cases when the carbon intensity is very low, and hence, the high operational carbon footprint during cold start on newer generation hardware is less significant compared to embodied carbon on older hardware during longer keep-alive. Our results demonstrate one such case where leveraging older-generation hardware does not always necessarily lead to a lower carbon footprint. The inverted case is shown in Fig. 3 (bottom) where keeping the DNA-visualization function alive on the older hardware for a longer period improves the service time as before, but may not result in carbon footprint saving – as alluded earlier, this is because of the impact of temporal variations in the carbon intensity and its impact on the operational carbon footprint. The inversion depends on many factors including carbon intensity, keep-alive period, execution length, cold-start, energy consumption of function, etc., and hence, the inversion point can vary among functions. Energy consumption is different in both scenarios (case A& B) because case B has longer service time due to cold-start and the DRAM energy may contribute toward the carbon footprint in different amounts for different functions.

In such situations, naívely choosing older hardware with a longer keep-alive period does not automatically lead to savings in the carbon footprint and service time and this is why the optimization is challenging. *Choosing keep-alive periods effectively requires carefully considering the carbon intensity of the energy source and adapting to temporal variations of the carbon intensity – since carbon intensity affects the operational carbon footprint component of the overall carbon footprint (embodied plus operational carbon footprint).*

Joint Optimization for Carbon Footprint and Service Time. Figure 4 demonstrates the potential for reducing carbon footprint while decreasing service time within the stateless serverless computing environment. The CO_2-OPT solution represents the most optimal solution solely focused on minimizing carbon footprint, while the SERVICE-TIME-OPT solution represents the optimal solution for minimizing service time alone. The ORACLE solution, theoretically optimal, aims to co-optimize both carbon footprint and service time.

Note that these three solutions are impractical in real-world systems as they rely on brute-force methods to explore all possible choices, providing only an upper bound for the design reference for ECOLIFE. The ENERGY-OPT solution stands as the traditional and naive optimal solution only focused on minimizing energy consumption to indirectly minimize carbon footprint. Notably, although energy consumption primarily contributes to the operational carbon footprint, ENERGY-OPT solution is far from CO_2-OPT solution. This is because energy-aware solutions often overlook the significance of embodied carbon footprint and variations in carbon intensity.

Unfortunately, co-optimization of service time and carbon footprint is challenging as shown in Figure 4 where even the ORACLE solution is more than 7% far from the respective SERVICE-TIME-OPT and CO_2-OPT solutions. An effective approach for co-optimizing service time and carbon footprint should incorporate the changes in function invocations and carbon intensity. Therefore, it is essential to develop a scheduler capable of adapting to a dynamically changing serverless environment. ECOLIFE is inspired by these necessities.

IV. DESIGN OF ECOLIFE

In this section, we first formulate the objective function that ECOLIFE minimizes. Then, we present the key ideas behind the design of ECOLIFE.

A. Problem Formulation

The goal of ECOLIFE is to determine the most suitable location (older-generation hardware or newer-generation hardware) and keep-alive periods for serverless functions – to co-optimize both service time and the carbon footprint. As mentioned in Sec. II, the keep-alive period can influence function cold starts, which in turn impacts both service time and carbon footprint. Our optimization can be subdivided into three main components: service time, carbon footprint during function execution, and carbon footprint during the keep-alive period of functions. The following expression shows the general objective function for achieving the optimization goal.

$$\underbrace{\lambda_s \frac{E[S_{f_{l,k}}]}{S_{f_{\max}}}}_{\text{Service Time}} + \underbrace{\lambda_c \frac{E[SC_{f_{l,k}}]}{SC_{f_{\max}}}}_{\text{Carbon Footprint During Execution}} + \underbrace{\lambda_c \frac{KC_{f_{l,k}}}{KC_{f_{k_{\max}}}}}_{\text{Carbon Footprint During Keep-alive period}} \quad \underset{l \in L,\ k \in \text{KAT}}{\text{argmin}}$$

λ_s and λ_c are the adjustable parameters to determine the optimization weights on reducing service time and carbon footprint, respectively. $E[S_{f_{l,k}}]$ is the expected value of service time of function f, keeping alive on hardware l, with k keep-alive time period. $S_{f_{\max}}$ is the maximum service time (the function has a cold start and is executed on the older-generation hardware). Similarly, $E[SC_{f_{l,k}}]$ denotes the expected carbon footprint during the service time of function f when kept alive on hardware l for k keep-alive period. $SC_{f_{\max}}$ represents the maximum carbon footprint during service time. The service time and carbon footprint of the function f account for the service time and carbon generated by the additional latency and delay.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:50:19 UTC from IEEE Xplore. Restrictions apply.

The term $KC_{f_{l,k}}$ is the carbon footprint of the function f during the keep-alive period k on hardware l, $KC_{f_{k\max}}$ is the maximum carbon footprint during keeping function f alive (function is kept alive on newer-generation hardware).

ECOLIFE aims to simultaneously determine the keep-alive locations l (older-generation hardware or newer-generation hardware) and the keep-alive periods k (selected from a set of keep-alive period values) for all invoked functions in order to co-optimize all functions, minimizing the overall carbon footprint and service time. To achieve this optimization, the scheduler should have the following design properties: (a) Adaptability to variations in function invocation patterns and carbon intensity: ECOLIFE must be capable of responding to rapidly changing patterns of function invocations and fluctuations in carbon intensity within short time periods. (b) Co-optimization of all invoked functions: Scheduling one serverless function should consider the keep-alive choices of other functions due to limited memory resources. (c) Low decision-making overhead: ECOLIFE should have low overhead to handle large serverless function invocation loads efficiently.

B. Overview of ECOLIFE

ECOLIFE is the first design using multi-generation hardware and intelligently selected keep-alive period to minimize the carbon footprint while maintaining high performance. ECOLIFE consists of three key components: function warm pools, the Keeping-alive Decision Maker (KDM), and the Execution Placement Decision Maker (EPDM). These components operate in coordination with one another. ECOLIFE manages two warm pools that monitor functions that are kept alive in the memory of Docker containers running on hardware spanning two across both old and new generations. Each pool of kept-alive functions has a memory constraint, necessitating ECOLIFE to ensure that the combined memory usage of all functions kept alive in the warm pool does not exceed the maximum memory capacity available. When the user sends requests of serverless function invocations, ECOLIFE uses the Keeping-alive Decision Maker to decide the keep-alive time and keep-alive location for every invoked function. If the memory space of hardware is not enough to hold a bursty load of function invocations, ECOLIFE performs adjustments in the pool of kept-alive functions for better usage of the available memory for incoming new functions that need to be kept alive. Regarding the function execution, ECOLIFE determines where to execute functions based on the Execution Placement Decision Maker to minimize carbon footprint and service time. Next, we will discuss the detailed design of each of the components of ECOLIFE, and how they contribute toward meeting the desired design properties discussed previously.

C. ECOLIFE's Keeping-alive Decision Maker (KDM)

ECOLIFE's KDM uses Particle Swarm Optimization (PSO) to determine the keep-alive time of functions. Before going into the basics of PSO and our novel extensions on vanilla PSO to solve ECOLIFE's optimization, we discuss the reasons behind using PSO in ECOLIFE.

Why does ECOLIFE use PSO? (a) Even the vanilla PSO algorithm is efficient in terms of determining the keep-alive time of serverless functions and has low decision-making overhead [32], fulfilling one of the design properties of ECOLIFE. PSO can rapidly converge to global optima due to its exploration-exploitation balance. Other exploration-exploitation optimization methods, such as reinforcement learning, have a larger overhead and require offline training. This is because PSO relies on simple, pre-defined rules for updating particle positions rather than learning complex policies through trial and error. (b) Due to its strong exploration capabilities, PSO is well suited to perform online optimization. It can continuously adapt to changing conditions and provide near-optimal solutions in dynamic environments, which is needed in a serverless context (one of the desired design properties). In PSO, multiple particles jointly explore the search space, which helps it to converge quickly when variations in system conditions change the optimal solution. Other traditional searching algorithms, such as gradient descent, are slower to adapt to the variations and are usually stuck in the local optima. Deep learning approaches are also not suitable for real-time online optimizations due to high training overhead and training data requirements. (c) In comparison to other heuristic optimization algorithms, such as Artificial Bee Optimization or Grey Wolf Optimization, PSO needs minimal parameter tuning with just three parameters. In our evaluation, we measured that PSO can reduce the carbon footprint by 17.4% and service time by 7.2%, compared to the Genetic Algorithm (another closely related nature-inspired optimization technique) with crossover probability of 0.6, mutation probability of 0.01, and population size of 15. Additionally, PSO showed a 6.2% reduction in carbon footprint and a 13.46% decrease in service time compared to the Simulated Annealing algorithm, which was set with an initial temperature of 100, a stop temperature of 1, and a temperature reduction factor of 0.9. Next, we briefly discuss the basics of a vanilla PSO optimizer. Thereafter, our extensions on vanilla PSO make ECOLIFE more suitable in the context of keeping serverless functions alive.

Basics of Particle Swarm Optimization. Particle swarm optimization is a meta-heuristic optimization algorithm inspired by how bird flocks forage. The bird flock effectively locates the best position of food source by sharing information collectively to let other birds know their respective positions. Birds determine whether the position they found is the optimal one and also share information about the best positions of the entire flock. Eventually, the entire bird flock gathers around the best position of food source. PSO utilizes massless particles to simulate birds in a flock, each particle has two attributes: velocity vector and position vector. Velocity represents the speed of movement, and position represents the direction of movement. The quality of each particle's position is determined by the fitness score. At the start of PSO search, N particles will be distributed at random positions in the search space. Particles change their positions in accordance with the

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:50:19 UTC from IEEE Xplore. Restrictions apply.

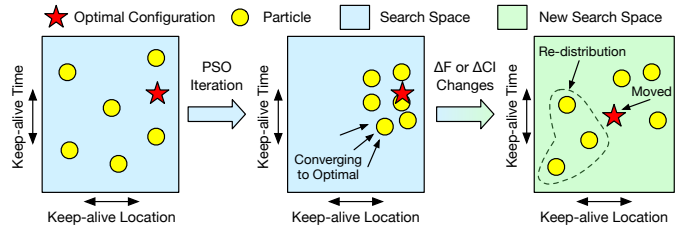

Fig. 5. *Optimization process for* ECOLIFE*'s Dynamic PSO (DPSO) – particles converge to the optimal after movement.*

following rules after each iteration:

$$\begin{cases} V_{t+1} = \omega * V_t + c_1 r_1 (X_{pbest} - X_t) + c_2 r_2 (X_{gbest} - X_t) \\ X_{t+1} = X_t + V_{t+1} \end{cases}$$

V_{t+1} and X_{t+1} represent the updated velocity and position of a particle, respectively, while V_t and X_t denote the previous velocity and position, respectively. X_{pbest} is the optimal position found by the individual particle, and P_{gbest} is the optimal position found by the entire swarm of particles. ω serves as the inertia weight, determining how much a particle should adhere to its previous velocity. c_1 and c_2 are cognitive and social coefficients, respectively, controlling the balance between refining the particle's search results and acknowledging the swarm's search results. r_1 and r_2 are random numbers uniformly distributed between 0 and 1. These adjustable coefficients regulate the trade-off between exploration and exploitation conducted by the swarm of particles. Next, we discuss the first extension that ECOLIFE performs on a vanilla PSO that helps it to quickly adjust to changing function invocation characteristics of serverless platforms.

ECOLIFE's Dynamic-PSO. ECOLIFE constructs a two-dimensional search space for each serverless function to determine the optimal position. One dimension represents two generations of hardware for keeping alive (l), while the other dimension covers a set of keep-alive times (k). For each new invocation of a serverless function, ECOLIFE assigns a PSO optimizer and preserves it in order to use it for the next function invocation consisting of parameters for keep-alive location and keep-alive time. However, a vanilla PSO algorithm is not ideally suited for achieving optimal solutions in the serverless environment due to the temporal fluctuations in carbon intensity and function invocations. Note that, configurable weights (w, c_1, and c_2) jointly regulate exploration and exploitation. One intuitive way is to dynamically adjust these weights based on the changes in carbon intensity and function invocation. The weights can be formally expressed as:

$$w = w_{\max} \left(\frac{\Delta F}{\Delta F_{\max}} + \frac{\Delta \mathrm{CI}}{\Delta \mathrm{CI}_{\max}} \right)$$

$$c_1 = c_2 = c_{\max} \left(1 - \frac{\Delta F}{\Delta F_{\max}} - \frac{\Delta \mathrm{CI}}{\Delta \mathrm{CI}_{\max}} \right)$$

Here, $\omega_{\max}$ denotes the maximum value of inertia weight, c_1 and c_2 share the same value, and $c_{\max}$ is the maximum value of the empirical coefficient. ΔF and $\Delta \mathrm{CI}$ denote the absolute changes of function invocations and carbon intensity,

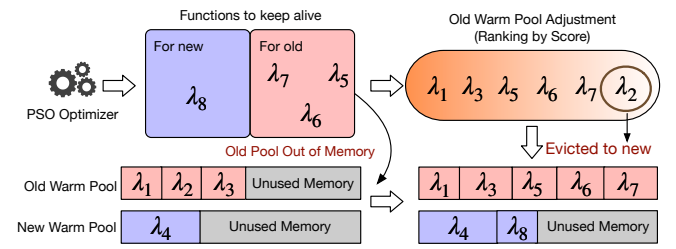

Fig. 6. ECOLIFE *strategically evicts functions from the warm pool when reaching the memory limit. Eviction is based on a priority score, evicted function is kept warm in the other generation's memory if there is enough space.*

respectively, since the last invocation. $\Delta F_{\max}$ and $\Delta \mathrm{CI}_{\max}$ are the maximum absolute changes in function invocations and carbon intensity across all observation windows so far.

To further enhance PSO's responsiveness to serverless environment variations, ECOLIFE introduces a *perception-response* mechanism to make PSO adapt to the dynamic environment (visually depicted in Fig. 5). This mechanism allows the particle swarm to be dynamically updated in response to environmental changes. If the perception indicates a change in the environment, the particle swarm updates to enlarge the exploration area. Conversely, if there is no perceived change in the environment, updates of the swarm are unnecessary. In ECOLIFE, perception is represented by changes in ΔF and $\Delta \mathrm{CI}$. ECOLIFE detects variations and divides the particle swarm into two halves. One-half is randomly redistributed within the search space, intensifying PSO's exploration and its ability to move past local optima. Meanwhile, the other half of the swarm retains its positions, providing the PSO optimizer with a level of memory, making it easier to find the optimal solution in dynamically changing environments.

ECOLIFE's Warm Pool Adjustment. After collecting all the keep-alive decisions generated by ECOLIFE's PSO, functions are designated for keeping alive on either old hardware, or new hardware, otherwise no keep-alive at all. Following these decisions, serverless functions will be kept alive in memory for the entire keep-alive period. However, incoming serverless functions may not be allocated due to memory limitation, despite their greater necessity for being kept alive. This can result in sub-optimal solutions. To address this issue, ECOLIFE adopts a priority eviction mechanism to sort functions already kept alive in the warm pool as well as those about to be kept alive to find the best arrangement (visually depicted in Fig. 6). This is performed by calculating the difference in service time and carbon footprint between cold start and warm start for functions on both old and new hardware to do priority ranking. After performing warm pool adjustment on one type of hardware, functions that are unable to be kept alive due to hardware memory limitation are transferred to another type of hardware for keeping alive, maximizing the utilization of hardware for keeping alive and thus increasing the probability of warm starts. Through warm pool adjustment, ECOLIFE co-optimizes all serverless functions to reduce service time and carbon footprint, which is the design objective of ECOLIFE.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:50:19 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 ECOLIFE's Scheduling Framework

Input: New invoked functions, initiate search space.
Output: Updated old warm pool p'_{old}, old warm pool p'_{new}.
Get the old warm pool p_{old} and new warm pool p_{new}.
for every new invoked function f **do**
    Execute function f based on the EPDM.
    Assign the Dynamic PSO optimizer.
    for following invocations of function f **do**
        Detect environment variations ΔF and ΔCI.
        Perform particle re-distribution and movement.
if p_{old} or p_{new} out of memory **then**
    Perform warm pool adjustment, update p_{old}, p_{new}.
    Keep functions alive based on updated warm pools.
else Keep function alive and update p_{old}, p_{new}.

D. ECOLIFE's Execution Placement Decision Maker

ECOLIFE uses the Execution Placement Decision Maker (EPDM) to determine where to execute this function. If the Docker containers on hardware retain this function, it implies that regardless of where the function is executed, it will receive a warm start. EPDM will execute this function on this hardware to avoid the cold start overhead. Else, if this function is not kept alive on only hardware, EPDM's decision will be based on the following scores to determine the optimal execution location:

$$f_{\text{score}} = \lambda_s \frac{S_r}{S_{f_{\text{max}}}} + \lambda_c \frac{SC_r}{SC_{\text{max}}}$$

Here, r denotes the execution location for the function f. $S_{f_{\text{max}}}$ and SC_{max} denotes the maximum service time and carbon footprint.

E. ECOLIFE: Combining All Design Elements

When a new serverless function is invoked for the first time, ECOLIFE makes the decision to allocate functions for execution (cold starts), and assign a PSO optimizer for this serverless function. PSO optimizer forms the search space for the function and initializes a number of particles randomly distributed in the space. As a function gets invoked multiple times, ECOLIFE utilizes the EPDM to determine where to execute this function to avoid cold start overhead based on the warm pools.

After execution, ECOLIFE's PSO detects the changes in the serverless environment (carbon intensity and function invocations), and the optimizer belonging to this function updates the PSO weights (ω, c_1, c_2) according to the changes and randomizes half of the swarm to explore in the search space.

After performing the particle movement, ECOLIFE uses the global best position generated by the PSO to decide the keep-alive location and keep-alive period. If there is limited memory for keep-alive, ECOLIFE performs warm pool adjustment for function keep-alive arrangement. Algorithm 1 summarizes the scheduling framework of ECOLIFE. ECOLIFE meets all the potential design properties as discussed in Sec. IV-A.

V. METHODOLOGY

Experimental Setup. ECOLIFE evaluation uses two types of testing nodes, `i3.metal` and `m5zn.metal`, which are selected from the AWS servers. `i3.metal` comprises a 2016 released 36 cores Xeon E5-2686 CPU, and a 2018 released 512 GiB Micron DRAM. `m5zn.metal` equips with a 2020 released 24 cores Xeon Platinum 8252C CPU, and a 2019 released 192 GiB Micron DRAM. This hardware configuration corresponds to Pair A in Table I, used as the default configuration in Sec. VI. Serverless functions are executed in the Docker container, as Docker is widely used in serverless execution [33]. Additionally, ECOLIFE uses an Intel Skylake-SP server with 16 cores, 64 GB memory, and a 4 Gbps network bandwidth as the controller node. We store each serverless function as a Docker image in an S3 bucket. The Docker image will be downloaded from the S3 bucket to the testing node assigned by ECOLIFE in the control node when execution starts during the simulation campaign.

ECOLIFE PSO Setup and Configuration. We deploy the PSO-based ECOLIFE on the Intel Skylake-SP server as previously discussed. We assign equal weights to both λ_s and λ_c ($\lambda_s = \lambda_c = 0.5$) to ensure equal optimization of service time and carbon footprint. As for the PSO in ECOLIFE, ω ranges from 0.5 to 1, c_1 ranges from 0.3 to 1 and c_2 ranges from 0.3 to 1. These weights jointly control the exploration and exploitation of PSO, as discussed in Section IV. We use 15 particles in the PSO, the number of particles influences the decision-making overhead, and changing the number of particles has negligible influence on the optimization results.

Evaluated Workloads. Serverless functions are collected from SeBS benchmark suites [28], including various scientific serverless workloads. These functions are invoked following the Microsoft Azure trace [26]. During our trace-driven simulation evaluation, the functions in the Microsoft Azure trace are selected for invocation randomly, but uniformly to ensure representativeness. ECOLIFE maps all serverless functions to the closest match, considering the memory and execution time.

Carbon Footprint Estimation and Carbon Intensity. ECOLIFE follows the carbon estimation mentioned in Sec. II. ECOLIFE uses a publicly available dataset [34] and well-established calculation methodologies [25] to determine the total embodied carbon of CPU and DRAM. ECOLIFE uses a typical four-year lifetime [35], [36] for DRAM and CPU. Carbon intensity within ECOLIFE is gathered from a widely-used Electricity Maps [37], and expanded to minute intervals to capture the temporal environmental variations. ECOLIFE primarily utilizes carbon intensity from California Independent System Operator (CISO), where carbon intensity fluctuates by an average of 6.75% hourly, with a standard deviation of 59.24. Additionally, ECOLIFE collects carbon intensity from Tennessee (TEN), Texas (TEX), Florida (FLA), and New York (NY) for robustness analysis. The ECOLIFE utilizes Likwid [38] - a simple Linux-based tool suite to read out RAPL [39] energy information and get info about turbo

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:50:19 UTC from IEEE Xplore. Restrictions apply.

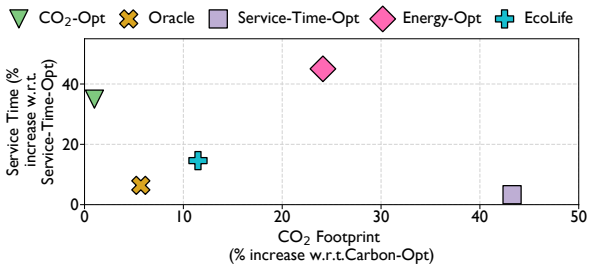

Fig. 7. ECOLIFE *is closest to the Oracle compared to other relevant techniques.*

mode steps on bare-metal machines for energy consumption measurement.

Relevant and Complementary Techniques. ECOLIFE is evaluated to compare with the following schemes:

NEW-ONLY, OLD-ONLY. NEW-ONLY, OLD-ONLY follow a ten (10) minutes keep-alive policy of OpenWhisk [33]. The NEW-ONLY scheme prioritizes the utilization of faster, newer hardware for executing functions under high-performance demands. The OLD-ONLY scheme operates in the opposite manner, it always utilizes older-generation hardware for executing functions. It is important to note that utilizing multi-generation hardware to keep functions alive is not a feature introduced in either the NEW-ONLY or OLD-ONLY scheme.

CO_2-OPT, SERVICE-TIME-OPT, and ORACLE. ECOLIFE compares against infeasible solutions, including CO_2-OPT (Carbon Footprint Optimal Solution), SERVICE-TIME-OPT (Performance Optimal Solution) and ORACLE (Best Optimal Solution). These solutions utilize heterogeneous hardware and present the theoretical upper bounds, which are computed via brute-forcing every possible scheduling option for each function invocation.

ECO-NEW, ECO-OLD. These schemes are static versions of ECOLIFE, and we use single-generation hardware to schedule functions. ECO-NEW and ECO-OLD primarily emphasize the determination of keep-alive periods while overlooking the trade-off between older hardware and newer hardware, which is the highlight brought by multi-generation hardware that ECOLIFE concentrates.

Figures of Merit. Carbon footprint and service time are two metrics used to evaluate ECOLIFE. They are represented as percentages under the SERVICE-TIME-OPT and CO_2-OPT (ORACLE in robustness analysis) to show the increase.

VI. EVALUATION

In this section, we evaluate the effectiveness of ECOLIFE, explain its effectiveness, and demonstrate its robustness.

A. *Effectiveness of* ECOLIFE

In Fig. 7, ECOLIFE stands out as the closest scheme to the ORACLE in terms of carbon footprint and service time among all the schemes. Recall from Sec. II, CO_2-OPT, SERVICE-TIME-OPT and ENERGY-OPT only minimize carbon footprint, service time and energy consumption respectively, and all of them are significantly far away from the ORACLE.

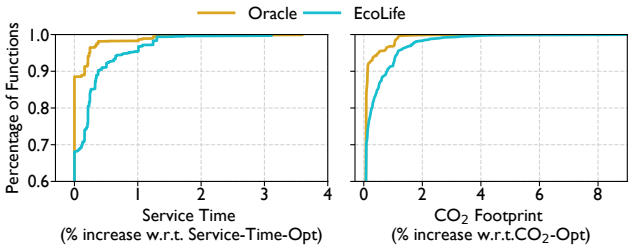

Fig. 8. *Cumulative distribution function for the carbon footprint and service time per function invocation of* ECOLIFE *stays close to the* ORACLE *for each function invocation.*

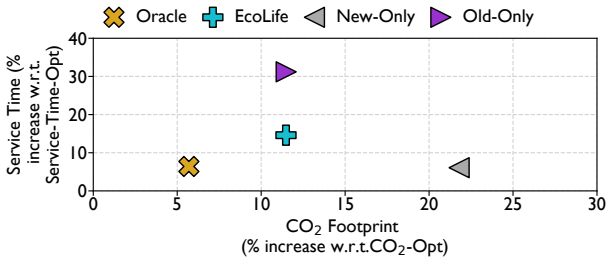

Fig. 9. ECOLIFE *outperforms single-generation only solutions.*

However, implementing the ORACLE directly in real-world systems is impractical. ECOLIFE co-optimize both metrics, while bridging the gap between sustainability and execution performance. Compared to ORACLE, ECOLIFE experiences a 7.7% increase in average service time and a 5.5% increase in average carbon footprint, respectively. Furthermore, ECOLIFE is close to ORACLE from the perspective of individual function invocations. As shown in Fig. 8, we present the cumulative distribution function (CDF) of service time and carbon footprint respectively, and both service time and carbon footprint remain less than 1% for each percentile of invoked functions. The service time is the average service time which includes queuing delay, setup delay, cold start (if applicable), and execution time. The P95 latency of ECOLIFE is within 15% increase of the service time in ORACLE. ECOLIFE decision-making overhead is also low and practical, less than 0.4% of service time, and 1.2% of carbon footprint for the invocation loads in the Azure trace. ECOLIFE achieves scalability by addressing the memory limitation problem of the co-located function with the warm pool adjustment.

As discussed in Sec. II, the utilization of multi-generation hardware can bring a high-performance and environmentally friendly serverless execution. In Fig. 9, We compare ECOLIFE with OLD-ONLY and NEW-ONLY in Sec. V with single-generation hardware under the 10-minute fixed keep-alive policy. While adopting older-generation hardware may reduce carbon emission, ECOLIFE's utilization of multi-generation hardware results in a service time saving of 12.7%. Similarly, although using newer-generation hardware will slightly accelerate the function execution, ECOLIFE can reduce carbon by 8.6% with multi-generation hardware. ECOLIFE is closer to ORACLE because of its heterogeneity and intelligently selected keep-alive periods. This confirms that ECOLIFE combines the

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:50:19 UTC from IEEE Xplore. Restrictions apply.

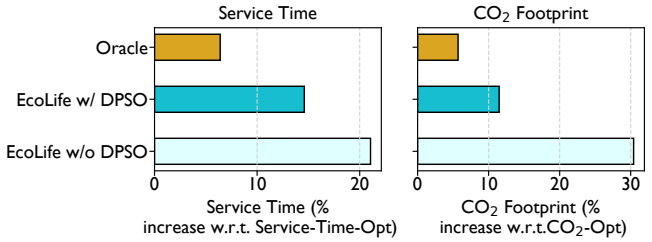

Fig. 10. *Dynamic PSO (DPSO) improves the effectiveness of* ECOLIFE, *especially in the terms of carbon footprint.*

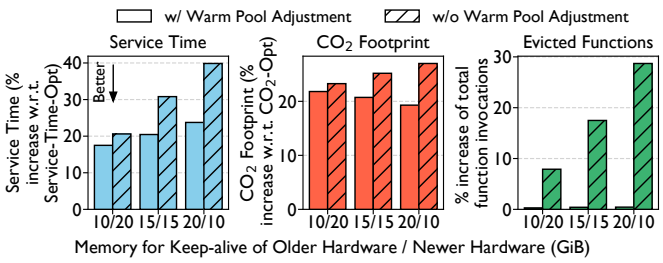

Fig. 11. ECOLIFE*'s warm pool adjustment strategy is key to its effectiveness.*

advantages of both hardware generations to achieve the co-optimization of service time and carbon footprint.

B. Reasons Behind ECOLIFE*'s Effectiveness*

ECOLIFE utilizes a perception-response mechanism to dynamically adapt to the PSO search space based on the changes in function invocations (ΔF) and carbon intensity (ΔCI), as described in Sec. IV. This enables ECOLIFE to effectively and precisely locate the near-optimal solution. As shown in Fig. 10, ECOLIFE without dynamic PSO experiences a 5.6% increase in service time and a 16.9% increase in carbon footprint. The decisions generated by dynamic PSO impact both the keep-alive period and the keep-alive location, which in turn directly affect the cold start overhead and the carbon footprint. Consequently, without dynamic PSO, the sub-optimal decisions would result in increased service time and carbon footprint.

Warm pool adjustment in ECOLIFE makes ECOLIFE effective when memory resources are insufficient to handle numerous serverless functions (Sec. IV).

In Fig. 11, we present a comparison of the service time, carbon intensity, and number of evicted functions with and without warm pool adjustment. The old and new hardware keep-alive memory size varies across three combinations, denoted as "old/new". The evicted functions are a result of limited memory space in their designated keep-alive hardware. A higher number of evicted functions indicates that the hardware is not utilizing its full potential to keep functions alive, resulting in longer service time because of the more frequent cold starts. As shown in Fig. 11, service time, carbon footprint, and evicted functions with warm pool adjustment are consistently lower than without. For example, with 15GiB memory available on old and new hardware (15/15), warm pool adjustment can save 7.9% of service time, 3.7% of carbon footprint, and keep 17% more functions alive which would otherwise be evicted.

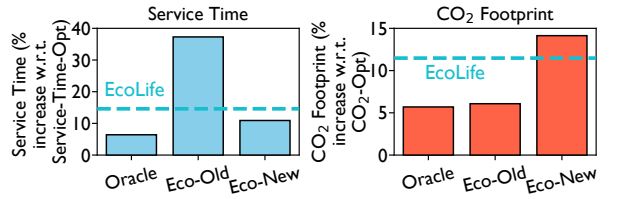

Fig. 12. ECOLIFE *can be applied to single-generation hardware, but using multi-generation hardware can co-optimize carbon emissions and service time.*

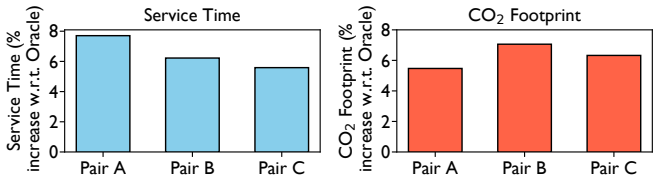

Fig. 13. ECOLIFE *is effective across different hardware pairs.*

C. Robustness of ECOLIFE

ECOLIFE is effective even with single-generation hardware (ECO-OLD or ECO-NEW), if multi-generation hardware is not available, as demonstrated in Fig. 12. Service time in ECO-OLD and carbon footprint in ECO-NEW are notably higher compared to the ORACLE. This is because ORACLE calculation is based on the multi-generation hardware, which achieves the best balance between minimizing service time and carbon footprint. However, implementing the ORACLE directly is impractical. ECOLIFE leverages the advantages of multi-generation utilization and enhances the co-optimization of service time and carbon footprint, offering a viable solution even in the absence of multi-generation hardware.

ECOLIFE is generally applicable to different hardware generation pairs, as we demonstrate its effectiveness against various hardware combinations from Table I in Fig. 13. Across all hardware generation pairs, ECOLIFE consistently achieves benefits close to the ORACLE, as both the service time and carbon footprint remain within a 7.5% margin to ORACLE. This demonstrates ECOLIFE's ability to flexibly leverage prior-generation hardware to balance service time and carbon footprint, without exploiting specific hardware types.

ECOLIFE's evaluation is focused on demonstrating its effectiveness for a single pair (two generations) to convey the benefit of its key insights. Assuming a five-year lifespan of one generation and alternate-year major hardware upgrades, one would likely expect to predominantly find two or three generations of hardware to be present at a given time in data centers. Three or more generations of hardware also present operational maintainability challenges. Nevertheless, ECOLIFE can work in the presence of multiple multi-generation pairs, by maintaining multiple warm pools.

We acknowledge that embodied carbon footprint can have small inaccuracy because the estimation relies on the accuracy

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:50:19 UTC from IEEE Xplore. Restrictions apply.

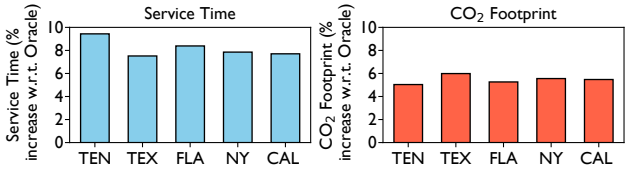

Fig. 14. ECOLIFE *remains effective and close to* ORACLE *across different geographical regions.*

of external data sources (e.g. vendor data) and the field is still rapidly evolving with multiple methodological practices. Nevertheless, the benefits of ECOLIFE remain within 7% (carbon) and 10% (service time) of ORACLE even if we allow a 10% estimation flexibility range for the embodied carbon footprint. While ECOLIFE primarily considers the embodied carbon footprint of CPU and DRAM, ECOLIFE is still effective when considering the embodied carbon footprint of other computer system components, including storage, motherboard, power unit, etc. ECOLIFE performs within 5.63% of ORACLE in carbon footprint and 8.2% in service time.

Finally, we evaluate ECOLIFE's effectiveness w.r.t. carbon intensity, as the carbon intensity profile may vary across geographical regions. In Fig. 14, we evaluate ECOLIFE using carbon intensity data from various regions. The results show that ECOLIFE remains effective across diverse geographical regions, as it remains within 7% of the Oracle in terms of service time and 6% in terms of carbon footprint. This showcases ECOLIFE's ability to adapt to various geographical environments and respond to trends in carbon intensity.

VII. DISCUSSION

We acknowledge that ECOLIFE's unorthodox approach of mixing multi-generational hardware raises several important and interesting considerations. For example, a critical consideration is operational maintainability in a data center environment. We argue that the presence of multi-generational hardware is already natural in today's data center because of portability, compatibility, smooth transition reasons, frequent hardware upgrades, and the need to support diverse customer needs (e.g., Amazon AWS). ECOLIFE simply exploits that opportunity for saving carbon footprint.

We also highlight that heterogeneity and multi-generation hardware have already been demonstrated to be beneficial from cost and performance perspectives for serverless workloads (e.g., IceBreaker [40]). ECOLIFE adds one more beneficial dimension – environmental sustainability to exploit these implicit investments/practices around multi-generation hardware. ECOLIFE advocates for longer lifetimes for older hardware. Thus, ECOLIFE opens the avenue for novel research to investigate the trade-off among performance, carbon footprint, lifetime, cost, and maintenance cost.

ECOLIFE's idea of lifetime extension for older hardware also has implications for the post-life/disposal carbon footprint of computing hardware. We also recognize that carbon footprint modeling and estimates are currently prone to errors, esp. for embodied carbon. As discussed earlier, ECOLIFE continues to provide benefits even for a range of estimations, but more efforts are needed to strengthen carbon footprint estimations.

VIII. RELATED WORK

Carbon footprint optimizations. The expansion of cloud and HPC infrastructure has spotlighted the importance of minimizing its carbon footprint, a concern echoed across numerous studies [1]–[9]. Efforts to reduce carbon emissions span diverse computation sources, from autonomous vehicles [41] and chip design [42] to smartphones [43] and the training of large language models [44], [45]. Within this context, ECOLIFE extends this effort into serverless computing, differentiating itself by optimizing the keep-alive strategy of serverless functions for reduced service time and carbon emissions. While cMemento [46] introduces carbon-aware memory placement in heterogeneous systems, it does not address the unique challenges of serverless function keep-alive that ECOLIFE tackles. Although previous research has proposed workload scheduling based on the carbon intensity's temporal and spatial variations [47]–[51], ECOLIFE innovates by considering serverless functions' execution and keep-alive on multi-generation hardware to promote sustainability.

Serverless function orchestration. Serverless computing has emerged as a scalable and efficient service model for cloud users [52], [53]. Research in this domain has extensively explored optimizations, focusing on cold start mitigation [54]–[58], hardware resource provisioning [59]–[64], stateful execution [15], [65]–[67], and cost-effectiveness [68]–[70]. Amid these developments, a gap remains in addressing the carbon footprint of serverless functions. While Icebreaker [40] and Molecule [71] have explored the use of heterogeneous hardware to enhance serverless function provisioning, they stop short of integrating carbon modeling to harness potential carbon savings. Similarly, energy-efficient solutions for serverless edge computing [72]–[74] underscore energy savings, which is only one aspect of system carbon footprint. In contrast with all prior works, ECOLIFE leverages an intelligent keep-alive mechanism on a multi-generation hardware platform, taking the first step towards sustainable serverless cloud computing.

IX. CONCLUSION

This paper presented ECOLIFE, a novel placement strategy designed to use multi-generation hardware for optimizing both the carbon footprint and service time of serverless functions. We hope our work will encourage the adaptation of multi-generation hardware in serverless execution environments, promoting more attention to computing sustainability and environmental considerations of large-scale computing systems.

Acknowledgment. We thank the reviewers for their constructive feedback. This work was supported by NSF Awards (2124897, and 1910601) and Northeastern University. This research partially used resources from the Massachusetts Green High Performance Computing Center (MGHPCC). We utilized ChatGPT, an AI language model developed by OpenAI, for partial assistance in drafting the text, and all generated content was thoroughly reviewed and edited for accuracy.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:50:19 UTC from IEEE Xplore. Restrictions apply.

References

[1] U. Gupta, Y. G. Kim, S. Lee, J. Tse, H.-H. S. Lee, G.-Y. Wei, D. Brooks, and C.-J. Wu, "Chasing carbon: The elusive environmental footprint of computing," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 854–867.

[2] U. Gupta, M. Elgamal, G. Hills, G.-Y. Wei, H.-H. S. Lee, D. Brooks, and C.-J. Wu, "Act: Designing sustainable computer systems with an architectural carbon modeling tool," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 784–799.

[3] L. Eeckhout, "A first-order model to assess computer architecture sustainability," *IEEE Computer Architecture Letters*, vol. 21, no. 2, pp. 137–140, 2022.

[4] A. Bersatti, E. Kim, and H. Kim, "Quantifying co 2 emission reduction through spatial partitioning in deep learning recommendation system workloads," *IEEE Micro*, 2024.

[5] C.-J. Wu, R. Raghavendra, U. Gupta, B. Acun, N. Ardalani, K. Maeng, G. Chang, F. Aga, J. Huang, C. Bai *et al.*, "Sustainable ai: Environmental implications, challenges and opportunities," *Proceedings of Machine Learning and Systems*, vol. 4, pp. 795–813, 2022.

[6] L. H. Kaack, P. L. Donti, E. Strubell, G. Kamiya, F. Creutzig, and D. Rolnick, "Aligning artificial intelligence with climate change mitigation," *Nature Climate Change*, vol. 12, no. 6, pp. 518–527, 2022.

[7] L. Lin and A. A. Chien, "Adapting datacenter capacity for greener datacenters and grid," in *Proceedings of the 14th ACM International Conference on Future Energy Systems*, 2023, pp. 200–213.

[8] B. Acun, B. Lee, F. Kazhamiaka, K. Maeng, U. Gupta, M. Chakkaravarthy, D. Brooks, and C.-J. Wu, "Carbon explorer: A holistic framework for designing carbon aware datacenters," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 118–132.

[9] B. Li, R. Basu Roy, D. Wang, S. Samsi, V. Gadepally, and D. Tiwari, "Toward sustainable hpc: Carbon footprint estimation and environmental implications of hpc systems," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–15.

[10] B. Li, Y. Jiang, and D. Tiwari, "Carbon in motion: Characterizing opensora on the sustainability of generative ai for video generation."

[11] B. Li, Y. Jiang, V. Gadepally, and D. Tiwari, "Toward sustainable genai using generation directives for carbon-friendly large language model inference," *arXiv preprint arXiv:2403.12900*, 2024.

[12] E. Jonas, J. Schleier-Smith, V. Sreekanti, C.-C. Tsai, A. Khandelwal, Q. Pu, V. Shankar, J. Carreira, K. Krauth, N. Yadwadkar *et al.*, "Cloud programming simplified: A berkeley view on serverless computing," *arXiv preprint arXiv:1902.03383*, 2019.

[13] J. M. Hellerstein, J. Faleiro, J. E. Gonzalez, J. Schleier-Smith, V. Sreekanti, A. Tumanov, and C. Wu, "Serverless computing: One step forward, two steps back," *arXiv preprint arXiv:1812.03651*, 2018.

[14] R. B. Roy, T. Patel, V. Gadepally, and D. Tiwari, "Mashup: making serverless computing useful for hpc workflows via hybrid execution," in *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2022, pp. 46–60.

[15] R. B. Roy, T. Patel, and D. Tiwari, "Daydream: executing dynamic scientific workflows on serverless platforms with hot starts," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 1–18.

[16] R. Basu Roy, T. Patel, R. Liew, Y. N. Babuji, R. Chard, and D. Tiwari, "Propack: Executing concurrent serverless functions faster and cheaper," in *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing*, 2023, pp. 211–224.

[17] R. Basu Roy and D. Tiwari, "Starship: Mitigating i/o bottlenecks in serverless computing for scientific workflows," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 8, no. 1, pp. 1–29, 2024.

[18] R. Chard, Y. Babuji, Z. Li, T. Skluzacek, A. Woodard, B. Blaiszik, I. Foster, and K. Chard, "Funcx: A federated function serving fabric for science," in *Proceedings of the 29th International symposium on high-performance parallel and distributed computing*, 2020, pp. 65–76.

[19] B. Carver, J. Zhang, A. Wang, A. Anwar, P. Wu, and Y. Cheng, "Wukong: A scalable and locality-enhanced framework for serverless parallel computing," in *Proceedings of the 11th ACM symposium on cloud computing*, 2020, pp. 1–15.

[20] T. J. Skluzacek, R. Chard, R. Wong, Z. Li, Y. N. Babuji, L. Ward, B. Blaiszik, K. Chard, and I. Foster, "Serverless workflows for indexing large scientific data," in *Proceedings of the 5th International Workshop on Serverless Computing*, 2019, pp. 43–48.

[21] R. B. Roy, T. Patel, and D. Tiwari, "Characterizing and mitigating the i/o scalability challenges for serverless applications," in *2021 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2021, pp. 74–86.

[22] J. Spillner, C. Mateos, and D. A. Monge, "Faaster, better, cheaper: The prospect of serverless scientific computing and hpc," in *High Performance Computing: 4th Latin American Conference, CARLA 2017, Buenos Aires, Argentina, and Colonia del Sacramento, Uruguay, September 20-22, 2017, Revised Selected Papers 4*. Springer, 2018, pp. 154–168.

[23] M. Malawski, A. Gajek, A. Zima, B. Balis, and K. Figiela, "Serverless execution of scientific workflows: Experiments with hyperflow, aws lambda and google cloud functions," *Future Generation Computer Systems*, vol. 110, pp. 502–514, 2020.

[24] Q. Jiang, Y. C. Lee, and A. Y. Zomaya, "Serverless execution of scientific workflows," in *International Conference on Service-Oriented Computing*. Springer, 2017, pp. 706–721.

[25] R. Lorenzini, "Digital & environment: How to evaluate server manufacturing footprint, beyond greenhouse gas emissions?" Nov 2021. [Online]. Available: https://boavizta.org/en/blog/empreinte-de-la-fabrication-d-un-serveur

[26] M. Shahrad, R. Fonseca, I. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *2020 USENIX annual technical conference (USENIX ATC 20)*, 2020, pp. 205–218.

[27] T. Yu, Q. Liu, D. Du, Y. Xia, B. Zang, Z. Lu, P. Yang, C. Qin, and H. Chen, "Characterizing serverless platforms with serverlessbench," in *Proceedings of the 11th ACM Symposium on Cloud Computing*, 2020, pp. 30–44.

[28] M. Copik, G. Kwasniewski, M. Besta, M. Podstawski, and T. Hoefler, "Sebs: A serverless benchmark suite for function-as-a-service computing," in *Proceedings of the 22nd International Middleware Conference*, 2021, pp. 64–78.

[29] N. Lövehagen, J. Malmodin, P. Bergmark, and S. Matinfar, "Assessing embodied carbon emissions of communication user devices by combining approaches," *Renewable and Sustainable Energy Reviews*, vol. 183, p. 113422, 2023.

[30] A. Shilov, "Sk hynix details ddr5-6400," Feb 2019. [Online]. Available: https://www.anandtech.com/show/13999/sk-hynix-details-its-ddr56400-dram-chip

[31] WikiChip, "14 nm lithography process." [Online]. Available: https://en.wikichip.org/

[32] G. Xu and G. Yu, "Reprint of: On convergence analysis of particle swarm optimization algorithm," *Journal of Computational and Applied Mathematics*, vol. 340, pp. 709–717, 2018.

[33] A. Kuntsevich, P. Nasirifard, and H.-A. Jacobsen, "A distributed analysis and benchmarking framework for apache openwhisk serverless platform," in *Proceedings of the 19th international middleware conference (posters)*, 2018, pp. 3–4.

[34] B. Davy, "Building an aws ec2 carbon emissions dataset," Feb 2024. [Online]. Available: https://medium.com/teads-engineering/building-an-aws-ec2-carbon-emissions-dataset-3f0fd76c98ac

[35] A. Shehabi, S. Smith, D. Sartor, R. Brown, M. Herrlin, J. Koomey, E. Masanet, N. Horner, I. Azevedo, and W. Lintner, "United states data center energy usage report," 2016.

[36] S. Govindan, A. Sivasubramaniam, and B. Urgaonkar, "Benefits and limitations of tapping into stored energy for datacenters," in *Proceedings of the 38th annual international symposium on Computer architecture*, 2011, pp. 341–352.

[37] E. Maps. (2024) Electricity Maps Live 24/7. [Online]. Available: https://app.electricitymaps.com/map

[38] J. Treibig, G. Hager, and G. Wellein, "Likwid: A lightweight performance-oriented tool suite for x86 multicore environments," in *Proceedings of PSTI2010, the First International Workshop on Parallel Software Tools and Tool Infrastructures*, San Diego CA, 2010.

[39] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen, and Z. Ou, "Rapl in action: Experiences in using rapl for power measurements," *ACM Transactions on Modeling and Performance Evaluation of Computing Systems (TOMPECS)*, vol. 3, no. 2, pp. 1–26, 2018.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:50:19 UTC from IEEE Xplore. Restrictions apply.

[40] R. B. Roy, T. Patel, and D. Tiwari, "Icebreaker: Warming serverless functions better with heterogeneity," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 753–767.

[41] S. Sudhakar, V. Sze, and S. Karaman, "Data centers on wheels: Emissions from computing onboard autonomous vehicles," *IEEE Micro*, vol. 43, no. 1, pp. 29–39, 2022.

[42] V. A. Chhabria, C. C. Sudarshan, S. Vrudhula, and S. S. Sapatnekar, "Towards sustainable computing: Assessing the carbon footprint of heterogeneous systems," *arXiv preprint arXiv:2306.09434*, 2023.

[43] J. Switzer, G. Marcano, R. Kastner, and P. Pannuto, "Junkyard computing: Repurposing discarded smartphones to minimize carbon," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 400–412.

[44] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[45] A. S. Luccioni, S. Viguier, and A.-L. Ligozat, "Estimating the carbon footprint of bloom, a 176b parameter language model," *Journal of Machine Learning Research*, vol. 24, no. 253, pp. 1–15, 2023.

[46] S. Köhler, B. Herzog, H. Hofmeier, M. Vögele, L. Wenzel, A. Polze, and T. Hönig, "Carbon-aware memory placement," in *Proceedings of the 2nd Workshop on Sustainable Computer Systems*, 2023, pp. 1–7.

[47] W. A. Hanafy, Q. Liang, N. Bashir, D. Irwin, and P. Shenoy, "Carbonscaler: Leveraging cloud workload elasticity for optimizing carbon-efficiency," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 7, no. 3, pp. 1–28, 2023.

[48] B. Li, S. Samsi, V. Gadepally, and D. Tiwari, "Clover: Toward sustainable ai with carbon-aware machine learning inference service," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2023, pp. 1–15.

[49] Z. Liu, M. Lin, A. Wierman, S. H. Low, and L. L. Andrew, "Greening geographical load balancing," *ACM SIGMETRICS Performance Evaluation Review*, vol. 39, no. 1, pp. 193–204, 2011.

[50] M. Chadha, T. Subramanian, E. Arima, M. Gerndt, M. Schulz, and O. Abboud, "Greencourier: Carbon-aware scheduling for serverless functions," in *Proceedings of the 9th International Workshop on Serverless Computing*, 2023, pp. 18–23.

[51] T. Anderson, A. Belay, M. Chowdhury, A. Cidon, and I. Zhang, "Treehouse: A case for carbon-aware datacenter software," *ACM SIGENERGY Energy Informatics Review*, vol. 3, no. 3, pp. 64–70, 2023.

[52] P. Castro, V. Ishakian, V. Muthusamy, and A. Slominski, "The rise of serverless computing," *Communications of the ACM*, vol. 62, no. 12, pp. 44–54, 2019.

[53] J. Wen, Z. Chen, X. Jin, and X. Liu, "Rise of the planet of serverless computing: A systematic review," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 5, pp. 1–61, 2023.

[54] E. Oakes, L. Yang, D. Zhou, K. Houck, T. Harter, A. Arpaci-Dusseau, and R. Arpaci-Dusseau, "{SOCK}: Rapid task provisioning with {Serverless-Optimized} containers," in *2018 USENIX annual technical conference (USENIX ATC 18)*, 2018, pp. 57–70.

[55] J. Cadden, T. Unger, Y. Awad, H. Dong, O. Krieger, and J. Appavoo, "Seuss: skip redundant paths to make serverless fast," in *Proceedings of the Fifteenth European Conference on Computer Systems*, 2020, pp. 1–15.

[56] Z. Li, L. Guo, Q. Chen, J. Cheng, C. Xu, D. Zeng, Z. Song, T. Ma, Y. Yang, C. Li *et al.*, "Help rather than recycle: Alleviating cold startup in serverless computing through {Inter-Function} container sharing," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, 2022, pp. 69–84.

[57] A. Fuerst and P. Sharma, "Faascache: keeping serverless computing alive with greedy-dual caching," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 386–400.

[58] D. Du, T. Yu, Y. Xia, B. Zang, G. Yan, C. Qin, Q. Wu, and H. Chen, "Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 467–481.

[59] A. Sahraei, S. Demetriou, A. Sobhgol, H. Zhang, A. Nagaraja, N. Pathak, G. Joshi, C. Souza, B. Huang, W. Cook *et al.*, "Xfaas: Hyperscale and low cost serverless functions at meta," in *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 231–246.

[60] Y. Fu, L. Liu, H. Wang, Y. Cheng, and S. Chen, "Sfs: Smart os scheduling for serverless functions," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 1–16.

[61] Z. Jia and E. Witchel, "Nightcore: efficient and scalable serverless computing for latency-sensitive, interactive microservices," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 152–166.

[62] H. Yu, H. Wang, J. Li, X. Yuan, and S.-J. Park, "Accelerating serverless computing by harvesting idle resources," in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 1741–1751.

[63] M. Bilal, M. Canini, R. Fonseca, and R. Rodrigues, "With great freedom comes great opportunity: Rethinking resource allocation for serverless functions," in *Proceedings of the Eighteenth European Conference on Computer Systems*, 2023, pp. 381–397.

[64] Z. Zhou, Y. Zhang, and C. Delimitrou, "Aquatope: Qos-and-uncertainty-aware resource management for multi-stage serverless workflows," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2022, pp. 1–14.

[65] S. Shillaker and P. Pietzuch, "Faasm: Lightweight isolation for efficient stateful serverless computing," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, 2020, pp. 419–433.

[66] H. Ding, Z. Wang, Z. Shen, R. Chen, and H. Chen, "Automated verification of idempotence for stateful serverless applications," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 2023, pp. 887–910.

[67] Z. Jia and E. Witchel, "Boki: Stateful serverless computing with shared logs," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, 2021, pp. 691–707.

[68] Y. Zhang, Í. Goiri, G. I. Chaudhry, R. Fonseca, S. Elnikety, C. Delimitrou, and R. Bianchini, "Faster and cheaper serverless computing on harvested resources," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, 2021, pp. 724–739.

[69] G. Sadeghian, M. Elsakhawy, M. Shahrad, J. Hattori, and M. Shahrad, "{UnFaaSener}: Latency and cost aware offloading of functions from serverless platforms," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, 2023, pp. 879–896.

[70] S. Eismann, L. Bui, J. Grohmann, C. Abad, N. Herbst, and S. Kounev, "Sizeless: Predicting the optimal size of serverless functions," in *Proceedings of the 22nd International Middleware Conference*, 2021, pp. 248–259.

[71] D. Du, Q. Liu, X. Jiang, Y. Xia, B. Zang, and H. Chen, "Serverless computing on heterogeneous computers," in *Proceedings of the 27th ACM international conference on architectural support for programming languages and operating systems*, 2022, pp. 797–813.

[72] P. Patros, J. Spillner, A. V. Papadopoulos, B. Varghese, O. Rana, and S. Dustdar, "Toward sustainable serverless computing," *IEEE Internet Computing*, vol. 25, no. 6, pp. 42–50, 2021.

[73] M. S. Aslanpour, A. N. Toosi, M. A. Cheema, and R. Gaire, "Energy-aware resource scheduling for serverless edge computing," in *2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 2022, pp. 190–199.

[74] A. Byrne, Y. Pang, A. Zou, S. Nadgowda, and A. K. Coskun, "Microfaas: Energy-efficient serverless on bare-metal single-board computers," in *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2022, pp. 754–759.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:50:19 UTC from IEEE Xplore. Restrictions apply.

Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper's Main Contributions

C_1 EcoLife is the first work leveraging multi-generation hardware, including older-generation hardware and newer-generation hardware, to reduce carbon emissions while achieving high performance. It includes three key components: function warm pools, the Keeping-alive Decision Maker (KDM), and the Execution Placement Decision Maker (EPDM).

C_2 EcoLife leverages a Particle Swarm Optimization (PSO) based algorithm to determine the placement of serverless functions and ensure a balance between the carbon footprint and service time of serverless functions. EcoLife extends the PSO with a perception-response mechanism to detect the variations of function invocation patterns and carbon intensity.

C_3 EcoLife introduces the function warm pool adjustment mechanism into the PSO optimizer design. EcoLife can adapt to the limited memory resources for keeping functions alive scenarios and improve the utilization of hardware resources.

B. Computational Artifacts

A_1 https://zenodo.org/records/10976139

Artifact ID	Contributions Supported	Related Paper Elements
A_1	C_1, C_2, C_3	Figure 5 Figure 6 Figure 7 Figure 8 Figure 9 Table 1 Algorithm 1

II. Artifact Identification

A. Computational Artifact A_1

Relation To Contributions

The codebase for EcoLife's framework contained in this artifact can be used to demonstrate that the utilization of multi-generation hardware can save carbon and maintain high performance aligning with the C_1. Each component outlined in C_1 is present in the EcoLife repository. The scripts also contain an optimizer, which is a dynamic version of the PSO optimizer incorporating the perception-response mechanism. it can provide the function placement decisions in response to changes in the serverless execution environment. This property corresponds to the contribution C_2. Moreover, this artifact can handle scenarios in which the memory for keep-alive is not enough for the serverless functions. To address this issue, this artifact includes code for adjusting warm pools for ongoing keep-alive functions, as described in contribution C_3.

Expected Results

We use an Oracle technique to compare with EcoLife. EcoLife achieves within 7.7% and 5.5% difference compared with the Oracle solution's service time and carbon footprint, respectively. Also, it can record every decision of each serverless function and its corresponding carbon footprint and service time. EcoLife can significantly save both desired metrics, which is the contribution of EcoLife. In addition, EcoLife stays effective using different carbon intensities.

Expected Reproduction Time (in Minutes)

The overall time for EcoLife simulation campaign is approximately 1700 minutes to process 2 weeks' data, including 1000 traces, 10^7 serverless function invocations, on a 16-core Intel Skylake-SP server platform. The computational time for setting up dependencies and warm pool adjustment can be ignored. The primary contributor to the overall execution time is the overhead incurred by PSO-based decision-making processes. The SC '24 AE process required the experiments to be finished within 8 hours and only considered such artifacts for providing all three reproducibility badges (link), but EcoLife reproducibility campaign is expected to take much longer to produce representative results.

Artifact Setup (incl. Inputs)

Hardware: EcoLife's measurement data to drive its simulation evaluation is collected on 2 testing nodes, `i3.metal` and `m5zn.metal`, which are selected from the AWS servers. `i3.metal` comprises a 2016 released 36 cores Xeon E5-2686 CPU, and a 2018 released 512 GiB Micron DRAM. `m5zn.metal` equips with a 2020 released 24 cores Xeon Platinum 8252C CPU, and a 2019 released 192 GiB Micron DRAM. Additionally, EcoLife uses `i3.2xlarge` selected from AWS, an Intel Skylake-SP server with 16 cores, 64 GB memory, and a 4 Gbps network bandwidth as the controller node.

Software: EcoLife can be executed with Python 3.8, and requires Python packages, including boto3, pandas, fire, shutil, random, numpy, json and csv for execution. EcoLife uses SSH for data collection of the service time and sends requests to the testing nodes. Energy consumption is measured using the Likwid. Serverless functions are executed in the Docker container on these testing nodes.

Datasets / Inputs: Serverless functions are collected from SeBS benchmark suites. These functions are invoked following the invocation traces provided by the Microsoft Azure Cloud. EcoLife maps all functions to the closest matching traces, considering the memory and execution time of each function.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:50:19 UTC from IEEE Xplore. Restrictions apply.

Installation and Deployment: First, establish the testing nodes (bare-metal machines) and the control nodes, ensuring the required code files are uploaded onto them. Then, verify the setup of AWS and obtain the AWS keys to integrate with the ECOLIFE for collecting the carbon footprint and service time. When simulation begins, ECOLIFE will make decisions regarding execution and keeping functions alive based on the profiling data.

Artifact Execution

Serverless functions are converted to Docker images and stored in the AWS S3 buckets. Control and test nodes are initialized to begin measuring carbon footprint and service time. Then, serverless functions are loaded into the containers, and users input function traces to start the simulation. ECOLIFE sets up two warm pools to monitor serverless functions and memory changes. After initialization, ECOLIFE uses the Keeping-alive Decision Maker to decide the keep-alive time and keep-alive location for every invoked function. Data on carbon footprint and service time are recorded for analysis, though keep-alive carbon footprint may be delayed until the next invocation. After making keep-alive decisions, ECOLIFE starts the warm pool adjustment to adjust the serverless functions that will be kept alive. ECOLIFE then decides function execution locations to minimize carbon footprint and service time. Execution-related carbon footprint and time are recorded. Finally, upon the completion of the overall process, all data is converted into a JSON file for analysis.

Artifact Analysis (incl. Outputs)

The program continuously monitors the carbon emissions and service time (execution time plus cold start overhead) of each function execution. These data are systematically recorded to analyze the average carbon footprint and service time. This evaluation compares the carbon footprint and service time efficiencies achieved by ECOLIFE and the ideal solution represented by the ORACLE.

Authorized licensed use limited to: Bibliothèque ÉTS. Downloaded on March 03,2026 at 00:50:19 UTC from IEEE Xplore. Restrictions apply.